SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – DEIGN AND CONNECTIVITY

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 1 – DEIGN AND CONNECTIVITY
Weightage:		**Total Marks:**	40 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Student Name:	**Sandra Sudevan**	**Reg. No.:**	**192424422**
Section / Batch:	**2024**	**Date of Submission:**	**1/08/2026**
Faculty In-charge:	**Dr. B.SHYAMALA GOWRI**	**Project Mode:**	**Individual Submission**

Code of Conduct

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sandra Sudevan

Requirement Analysis (5 Marks)	ER Diagram Design (10 Marks)	Table Design & Normalization (5 Marks)	Database Connectivity using Python (20 Marks)	CRUD Operations (5 Marks)	Total (35 Marks)

Scenario 1: University Student Management System

A university manages information related to students, faculty members, departments, courses, and examinations. Currently, student information is stored in spreadsheets by different departments, resulting in duplicated records, inconsistent student IDs, and difficulties in generating reports. The university administration wants to develop a centralized database system that can store student details, course enrollments, faculty assignments, attendance records, and examination results. Students should be able to register for multiple courses, while each course may have multiple students enrolled. Faculty members may teach several courses within a semester. The management also requires reports such as department-wise student strength, faculty workload, and student performance statistics. Design an appropriate database schema by identifying entities, attributes, primary keys, foreign keys, and relationships. Connect the database using Python and demonstrate CRUD operations for managing student records.

Scenario 2: Hospital Patient Record Management System

A multi-specialty hospital receives thousands of patients every month. Currently, patient records are maintained manually, making it difficult to track patient history, doctor consultations, prescriptions, laboratory tests, and billing information. The hospital management wants a database-driven application that can store patient details, doctor information, appointments, treatment records, and payment details. Each patient may visit multiple doctors over time, and doctors may consult many patients. Laboratory tests and prescriptions must also be linked to patient records. The system should provide quick access to patient histories and generate reports on disease trends and doctor workloads. Design a relational database schema for this system and use Python SQL libraries to connect to the database and manage patient information efficiently.

Scenario 3: E-Commerce Online Shopping Platform

An online shopping company sells products across different categories such as electronics, clothing, books, and household items. Customers create accounts, browse products, place orders, and make online payments. Each order may contain multiple products, and inventory levels must be updated automatically after each purchase. The company also wants to track customer reviews and ratings for products. The management requires reports on top-selling products, customer purchase patterns, and monthly revenue. Analyze the business requirements and design a database containing tables for customers, products, orders, payments, and reviews. Establish relationships among the tables and develop Python programs to perform database connectivity and CRUD operations.

Scenario 4: Library Management System

A large academic library manages thousands of books, journals, magazines, and digital resources. Students and faculty members borrow and return books regularly. The current manual system often results in misplaced records and inaccurate tracking of issued books. The library wants to automate its operations by maintaining information about books, authors, publishers, members, and transactions. A member may borrow multiple books, and a book may be issued to different members over time. The system should also calculate fines for overdue books and generate inventory reports. Design an efficient database schema and implement database connectivity using Python to manage book records and member transactions.

Scenario 5: Hotel Reservation and Booking System

A hotel chain operates multiple branches across different cities. Customers can book rooms online or through customer service representatives. The hotel management wants a centralized system to manage customer information, room availability, reservations, payments, and employee details. Different room categories such as standard, deluxe, and suite must be maintained. The system should track check-in and check-out details, generate invoices, and maintain customer booking history. Analyze the requirements and design a database structure that minimizes redundancy and supports efficient querying. Use Python to connect to the database and demonstrate room booking, reservation updates, and customer management functionalities.

Scenario 6: Banking and Customer Account Management System

A commercial bank manages savings accounts, current accounts, loans, and customer transactions. Customers may own multiple accounts, and each account contains transaction records such as deposits, withdrawals, and transfers. The bank requires a secure database system to store customer information, account details, branch information, and loan records. Managers need reports on customer balances, transaction histories, and loan repayments. The system must support quick retrieval of information and ensure data consistency. Design the database schema, define relationships between customers and accounts, and implement Python-based database connectivity for account management operations.

Scenario 7: Food Delivery Application

A food delivery platform connects customers, restaurants, and delivery agents through a mobile application. Customers place orders from different restaurants, while delivery agents deliver food to specified locations. Restaurants maintain menus and pricing information. The platform must track orders, delivery status, payments, and customer feedback. Management wants analytical reports on popular restaurants, peak ordering times, and delivery performance. Design a database containing tables for customers, restaurants, menu items, orders, delivery agents, and payments. Implement Python-based CRUD operations to manage customer orders and delivery records.

Scenario 8: Employee Payroll Management System

A manufacturing company employs hundreds of workers and administrative staff. The Human Resources department currently manages employee information and salary calculations using spreadsheets, which often leads to payroll errors. The company wants a database-driven payroll system to store employee details, department information, attendance records, salary structures, and tax deductions. Each employee belongs to a department, and payroll must be generated monthly based on attendance and salary components. Design a normalized database schema and create a Python application that performs employee management and payroll processing using database connectivity.

Scenario 9: Online Learning Management System

An educational technology company provides online courses to students worldwide. Students enroll in courses, watch video lectures, submit assignments, and receive grades. Instructors create courses and manage learning materials. The platform needs a database system to maintain student profiles, instructor details, course catalogs, enrollments, assignments, and assessment results. Administrators require reports on student progress, course completion rates, and instructor performance.

Design a relational database schema that supports these requirements and implement Python programs for managing enrollments and academic records.

Scenario 10: Smart Transportation and Ticket Booking System

A transportation company operates buses across multiple cities. Passengers can search routes, book tickets, cancel reservations, and make online payments. The company must maintain information about buses, routes, schedules, drivers, passengers, and ticket bookings. Each bus operates on multiple routes, and passengers may make several bookings over time. Management wants to analyze passenger demand, route utilization, and revenue generation. Design a database system that efficiently stores transportation data and supports ticket booking operations. Use Python SQL libraries to connect to the database and perform CRUD operations for route management, passenger registration, and ticket reservations.

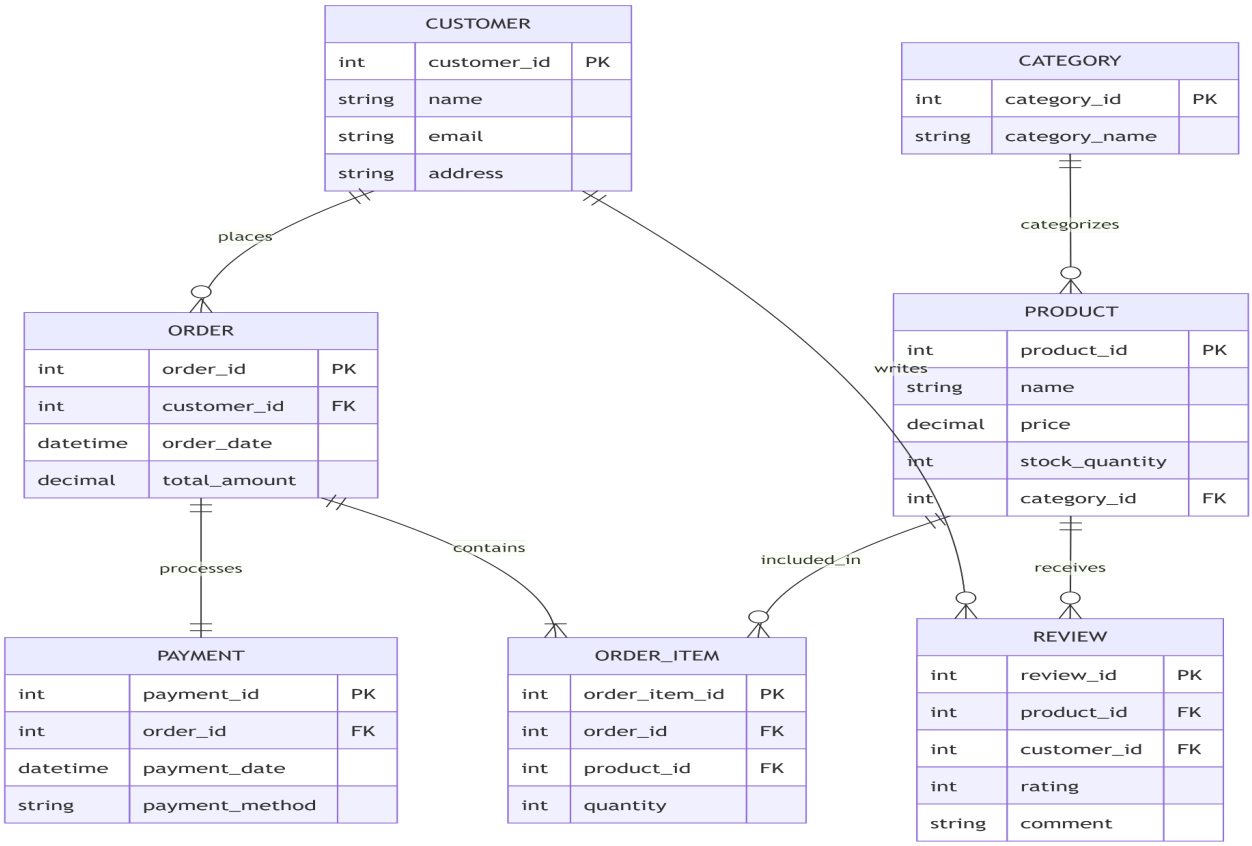
1. **Scenario 3: E-Commerce Online Shopping Platform**

An online shopping company sells products across different categories such as electronics, clothing, books, and household items. Customers create accounts, browse products, place orders, and make online payments. Each order may contain multiple products, and inventory levels must be updated automatically after each purchase. The company also wants to track customer reviews and ratings for products. The management requires reports on top-selling products, customer purchase patterns, and monthly revenue. Analyze the business requirements and design a database containing tables for customers, products, orders, payments, and reviews. Establish relationships among the tables and develop Python programs to perform database connectivity and CRUD operations.

Procedure

1. Open **VS Code** and create a Python project.
2. Install the MySQL Connector using `pip install mysql-connector-python`.
3. Create the `ecommerce_db` database and required tables in MySQL.
4. Write the Python program to connect to the MySQL database.
5. Implement CRUD operations (Insert, View, Update, Delete).
6. Run the program using `python crud.py`.
7. Verify the output and close the database connection.

ER DIAGRAM


CUSTOMER
int customer_id PK
string name
string email
string address
CATEGORY
int category_id PK
string category_name
places
categorizes
ORDER
int order_id PK
int customer_id FK
datetime order_date
decimal total_amount
PRODUCT
int product_id PK
string name
decimal price
int stock_quantity
int category_id FK
writes
processes
contains
included_in
receives
PAYMENT
int payment_id PK
int order_id FK
datetime payment_date
string payment_method
ORDER_ITEM
int order_item_id PK
int order_id FK
int product_id FK
int quantity
REVIEW
int review_id PK
int product_id FK
int customer_id FK
int rating
string comment

```
[notice] A new release of pip is available: 25.0.1 -> 26.2
[notice] To update, run: C:\Users\sandr\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n
2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\sandr\OneDrive\Desktop\ECommerce_Project> python crud.py

===== E-Commerce CRUD Operations =====
1. Add Customer
2. View Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter your choice (1-5): 1
Enter Customer ID: 101
Enter Customer Name: SANDRA
Enter Email: SANDRA@1811gmail.com
Enter Phone Number: 1234567891
Customer Added Successfully!
```

```
===== E-Commerce CRUD Operations =====
1. Add Customer
2. View Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter your choice (1-5): 2

Customer Details
-----------------------------------------------
(101, 'SANDRA', 'SANDRA@1811gmail.com', '1234567891')
```

```
===== E-Commerce CRUD Operations =====
1. Add Customer
2. View Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter your choice (1-5): 3
Enter Customer ID to Update: 102
Enter New Phone Number: 99999999999
Customer Updated Successfully!
```

```
===== E-Commerce CRUD Operations =====
1. Add Customer
2. View Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter your choice (1-5): 4
Enter Customer ID to Delete: 102
Customer Deleted Successfully!
```

```
===== E-Commerce CRUD Operations =====
1. Add Customer
2. View Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter your choice (1-5): 5
Thank You!
PS C:\Users\sandr\OneDrive\Desktop\ECommerce_Project>
```

Result:

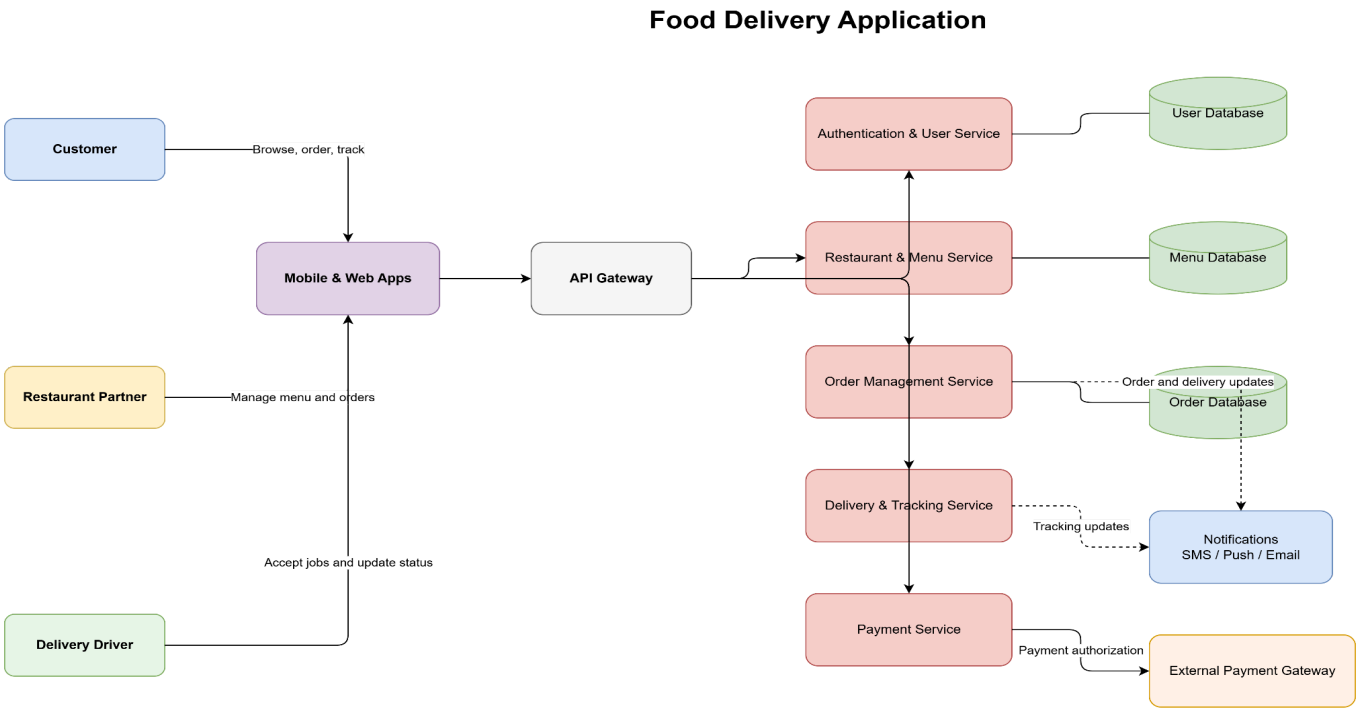
The E-Commerce database was successfully connected using Python, and CRUD operations were performed successfully.

2. Scenario 7: Food Delivery Application

A food delivery platform connects customers, restaurants, and delivery agents through a mobile application. Customers place orders from different restaurants, while delivery agents deliver food to specified locations. Restaurants maintain menus and pricing information. The platform must track orders, delivery status, payments, and customer feedback. Management wants analytical reports on popular restaurants, peak ordering times, and delivery performance. Design a database containing tables for customers, restaurants, menu items, orders, delivery agents, and payments. Implement Python-based CRUD operations to manage customer orders and delivery records.

Aim

To design and implement a **Food Delivery Application** database using **Python and MySQL**, establish database connectivity, and perform **CRUD (Create, Read, Update, Delete)** operations on customer orders and delivery records.



Procedure

1. Open **Visual Studio Code** and create a new Python project.
2. Install the MySQL Connector using the command pip install mysql-connector-python.
3. Create the food_delivery_db database and the Orders table in MySQL.
4. Write a Python program to connect to the MySQL database.
5. Implement CRUD operations to add, view, update, and delete order records.
6. Run the program using python crud.py.
7. Test each CRUD operation and verify the results.
8. Close the database connection after execution.

```
===== FOOD DELIVERY APPLICATION =====
1. Add Order
2. View Orders
3. Update Delivery Status
4. Delete Order
5. Exit
Enter your choice (1-5): 1
Enter Order ID: 111
Enter Customer Name: sandra
Enter Restaurant Name: velu
Enter Food Item: idly
Enter Quantity: 4
Enter Total Amount: 206
Enter Delivery Status: pending
Order Added Successfully!
```

```
===== FOOD DELIVERY APPLICATION =====
1. Add Order
2. View Orders
3. Update Delivery Status
4. Delete Order
5. Exit
Enter your choice (1-5): 2

------ ORDER DETAILS ------
(111, 'sandra', 'velu', 'idly', 4, Decimal('206.00'), 'pending')
```

```
===== FOOD DELIVERY APPLICATION =====
1. Add Order
2. View Orders
3. Update Delivery Status
4. Delete Order
5. Exit
Enter your choice (1-5): 3
Enter Order ID: 101
Enter New Delivery Status: delivered
Delivery Status Updated Successfully!
```

```
===== FOOD DELIVERY APPLICATION =====
1. Add Order
2. View Orders
3. Update Delivery Status
4. Delete Order
5. Exit
Enter your choice (1-5): 4
Enter Order ID to Delete: 101
Order Deleted Successfully!
```

```
===== FOOD DELIVERY APPLICATION =====
1. Add Order
2. View Orders
3. Update Delivery Status
4. Delete Order
5. Exit
Enter your choice (1-5): 5
Thank You!
PS C:\Users\sandr\OneDrive\Desktop\Food_Delivery_Project>
```

Result

The Food Delivery Application database was successfully connected using Python and MySQL, and CRUD operations on customer orders and delivery records were performed successfully.